SWE5001 Practice Project · Graduate Certificate in Architecting Scalable Systems

# Speaker Script and Live Demo Run Book

## BlocBuy — Team 8 · Wednesday 21 October 2026, 16:15–16:55 SGT (Zoom)

| | |
|---|---|
| **Responsible** | Lim Jiechao (presentation lead, #461); reviewer Liu Jin. |
| **Companion files** | `swe5001-presentation.html`: the deck, 24 slides plus 17 backup slides. Open it in Chrome or Edge. Press **S** for the notes window with this script and a pace timer (**T** starts it), **B** for the backup index, **O** for the overview, **F** for fullscreen. `swe5001-presentation.pdf` is the same deck as a PDF (41 pages: 24 main, then backups). Eric's spoken wording for segment 3 and the demo comes from `notes/speaker-script-eric.md`. |
| **Single source** | `src/content.js` holds wording, timings, cues, Q&A and open items; `src/slides-main.js` and `src/slides-backup.js` hold the slide layouts and `src/script.js` this document; the UML diagrams are shared from `docs/presentation-kit/diagrams/`. Run `npm ci` once in `docs/presentation-kit/`, then `npm run all` in `docs/swe5001/presentation/` to rebuild everything in `dist/`. |
| **Facts as of** | 19 Oct 2026, with the 20 Oct demo decision (no live load test). V3 ran on AWS; every other test is shown as "not run on AWS" with its local result. No gaps remain. |

**How to use this document**

- **Speakers:** read your segment (§1–§5) aloud against a timer at least twice. Each slide card shows the slot and how long the sample wording takes at 130 words a minute. Rephrase in your own words; never read the slide.
- **Demo team:** §6 is the run book: preconditions, screen layout, then every step with what to operate, what to say, what you should see and when to fall back. §7 adds Plan C.
- **Lead:** the run sheet and coverage checks below, the contingencies in §7, the timeline in §8 and the checks and open items in §9 are yours.

Structure follows "04 Project Presentation Guidelines & Schedule" V2.8 exactly: one chapter per segment (1 Overview, 2 Project Conduct, 3 Logical, 4 Physical, 5 DevOps, 6 Demo), each opening with the guideline's own sub-items.

# Run sheet and coverage check

## Slide-by-slide run sheet

| Clock | Slot | Slide | Title | Speaker | Item | Est. |
|---|---|---|---|---|---|---|
| 16:15 | 15 s | 1 | BlocBuy | Lim | 1a | 14 s |
| 16:15:15 | 40 s | 2 | Group buying only works if enough people join in time | Lim | 1a | 40 s |
| 16:15:55 | 40 s | 3 | A two-sided platform built around one interaction: forming a team | Lim | 1b | 39 s |
| 16:16:35 | 25 s | 4 | Scope: the full buying journey, on three services | Lim | 1c | 25 s |
| 16:18 | 45 s | 5 | Status: what is verified on AWS, and what is still open | Lim | 2a | 44 s |
| 16:18:45 | 45 s | 6 | On schedule, about 50 person-days across five people | Lim | 2b | 37 s |
| 16:20 | 90 s | 7 | Why three services, not five | Eric | 3a | 71 s |
| 16:21:30 | 80 s | 8 | Four decisions shape the logical design | Eric | 3a | 59 s |
| 16:22:50 | 90 s | 9 | Logical view: one direct call, everything else by events | Eric | 3b | 70 s |
| 16:24:20 | 75 s | 10 | Bounded contexts decide the services; events are the contract | Eric | 3c | 60 s |
| 16:25:35 | 85 s | 11 | How producers and consumers use the platform, and what reuse costs | Eric | 3c | 67 s |
| 16:29 | 85 s | 12 | Physical decisions: managed, disposable, low cost | Liu Jin | 4a | 73 s |
| 16:30:25 | 85 s | 13 | Physical view: private services behind CloudFront, across two zones | Liu Jin | 4b | 60 s |
| 16:31:50 | 45 s | 14 | Each service scales on its own signal, and how far it goes | Liu Jin | 4b | 42 s |
| 16:32:35 | 45 s | 15 | Each service owns its data | Liu Jin | 4c | 42 s |
| 16:33:20 | 95 s | 16 | Team expiry, end to end: one transaction in, one transaction out | Yorik | 4d | 79 s |
| 16:34:55 | 65 s | 17 | Failures are expected, and handled | Yorik | 4d | 48 s |
| 16:38 | 65 s | 18 | One main branch; production changes only by promotion | Pan | 5a | 38 s |
| 16:39:05 | 70 s | 19 | Every pull request is checked before it can merge | Pan | 5b | 51 s |
| 16:40:15 | 95 s | 20 | One promotion builds, deploys, tests and releases, then destroys it all | Pan | 5b | 67 s |
| 16:41:50 | 75 s | 21 | What the real cloud taught us | Pan | 5c | 43 s |
| 16:43:05 | 55 s | 22 | No standing keys: who and what can touch AWS | Pan | 5b | 47 s |
| 16:45 | 15 s | 23 | Live demo | Eric | 6a | 9 s |
| 16:53 | Q&A | 24 | Thank you | Lim | 6b | |

Segment talk budget vs slots vs estimated speaking time: §1 120 s / 120 s / 118 s · §2 90 s / 90 s / 81 s · §3 420 s / 420 s / 327 s · §4 420 s / 420 s / 344 s · §5 360 s / 360 s / 246 s · §6 480 s / 15 s / 9 s (§6 talk is the demo itself).

The wording fills about 70% of each slot; the rest is for pointing at the slide and a natural pace.

Q&A windows (inside the 40 minutes): 16:17–16:18 · 16:19:30–16:20 · 16:27–16:29 · 16:36–16:38 · 16:44–16:45 · 16:53–16:55. Background action: **16:44:30** screen-share switch Lim → Eric. No load test runs during the talk.

## Every guideline item has slides and a speaker

| Item | Guideline requirement | Slides | Speaker |
|---|---|---|---|
| **1a** | Explain the context and the business problem that the solution as a whole is trying to achieve. | 1, 2 | Lim |
| **1b** | Highlight the platform-thinking aspects of your solution. | 3 | Lim |
| **1c** | Explain the scope of the project. | 4 | Lim |
| **2a** | Explain the current status of the project including outstanding issues. | 5 | Lim |
| **2b** | Describe the actual project milestones and the rough total effort expended by each team member. | 6 | Lim |
| **3a** | Highlight the key architectural decisions that influence the logical architecture. | 7, 8 | Eric |
| **3b** | Explain the logical components of the solution by showing a logical detail deployment diagram and other supporting views. | 9 (UML logical deployment view) | Eric |
| **3c** | Where relevant, highlight the elements of Domain Driven Design that help to shape the design of the solution as a platform. In particular, describe how the platform is designed to interact with the producers and consumers. | 10, 11 | Eric |
| **4a** | Highlight the key architectural decisions that influence the physical architecture. | 12 | Liu Jin |
| **4b** | Describe the physical architecture that includes the network, nodes, technologies and cloud services that are used in the solution. | 13, 14 (UML deployment) | Liu Jin |
| **4c** | Describe the persistence design of the solution. | 15 | Liu Jin |
| **4d** | Describe the detailed design of an architecturally representative user case or user story. | 16, 17 (UML sequence) | Yorik |
| **5a** | Describe the strategy to manage the source code and artifacts. | 18 | Pan |
| **5b** | Describe the DevOps pipeline and procedures. | 19, 20, 22 | Pan |
| **5c** | Highlight any technical findings and issues encountered. | 21 | Pan |
| **6a** | Demonstrate some of the significant use cases of the solution. | 23 + demo step 1 | Eric |
| **6b** | Demonstrate how the solution deals with some of the important quality attributes e.g., scalability (such as demonstrating auto-scaling), performance (show using some scripted performance testing), maintainability (show/run the DevOps pipeline). | demo steps 2–4 + 24 | Eric, Pan, Lim |

## Every briefing question has an answer on a slide

| Briefing question or success criterion | Where |
|---|---|

| | |
|---|---|
| Q1 Pain points | 2 |
| Q2 Platform, common services, reusable frameworks | 3, 9, 11; B5, B10 |
| Q3 Benefit of building on the platform, demonstrated | 11 (measured reuse) |
| Q4 Scale, and proof | 14 (capacity statement), demo step 2; B12 |
| Q5 Technical and non-technical benefits, demonstrated | 12 (measured cost), 14, 19, demo |
| Q6 Value of cloud-native design, demonstrated | 12 (US$0.49 measured), 13, 20 |
| Q7 Which parts of the lifecycle are automated, and the effort saved | 19 (5,252 runs), 20 |
| Q8 Effort within the guideline | 6 |
| Success criterion: seed, producers, consumers | 3 |
| Success criterion: more seeds in future | 3 (future seeds line) |
| Success criterion: analytics | B11 (Q&A) |
| Module: robust API access; minimum security controls | 11, B10; 22 |

## Cue words in the team backchannel (never the Zoom chat)

| Message | Meaning | Sent by |
|---|---|---|
| `1 MIN` | Speaker has one minute of talk left in the segment | Lim |
| `TIME` | Stop talking; move to questions or the hand-over | Lim |
| `CLIP n` | Live step n failed; Liu Jin is ready to share the recording for step n | Eric → Liu Jin |
| `SHARE` | Next person: start screen sharing now | Lim |

# 1 Overview (2+1 minutes)

| | |
|---|---|
| **Clock** | 16:15:00 – 16:18:00 (2 min talk + 1 min Q&A; talk ends 16:17:00) |
| **Owner** | Lim Jiechao |
| **Slides** | 1, 2, 3, 4 |

**Guideline requirement (V2.8)**

**1a** — Explain the context and the business problem that the solution as a whole is trying to achieve.
**1b** — Highlight the platform-thinking aspects of your solution.
**1c** — Explain the scope of the project.

## Slide 1 BlocBuy

16:15:00–16:15:15 · 15 s

**Speaker** Lim · covers guideline item **1a**

***Say*** *Good afternoon, we're Team 8. This is BlocBuy, a group-buying platform built to scale on AWS. We'll cover the problem, the design and how we ship it, then show it live.*

≈ 31 words · ≈ 14 s at 130 wpm · slot 15 s

**Cues**

- Lim opens swe5001-presentation.html in Chrome, presses S for the notes window (keep it off the shared screen) and T there at 16:15:00. In Zoom, share only the deck window.
- Liu Jin watches ECS health on the dashboard, muted.

**Hand-over** *"So, what problem are we solving?"*

## Slide 2 Group buying only works if enough people join in time

16:15:15–16:15:55 · 40 s

**Speaker** Lim · covers guideline item **1a**

***Say*** *Group buying means people buy together to get a lower price. But it only works if enough people join before the deadline. Most groups run this in chat apps and spreadsheets. Buyers can't see how close a deal is. Organisers collect the money and do refunds by hand. And small sellers have nowhere to post a team price. With BlocBuy, you pay to join a team, the price drops as more people join, and if the team doesn't fill in time, everyone gets their money back automatically.*

≈ 87 words · ≈ 40 s at 130 wpm · slot 40 s

**Hand-over** *"And we built it as a platform, not a shop."*

### Slide 3 A two-sided platform built around one interaction: forming a team

16:15:55–16:16:35 · 40 s

**Speaker** Lim · covers guideline item **1b**

**Say** *At the centre is one thing: forming a team at a team price. Sellers put up offers, and buyers browse and join them. When buyers start a team or share an invite, they bring in new people, so buyers help grow the platform too. Every new member makes the deal better for everyone; that's our network effect. We start it off with a few hand-picked offers. Sign-in, notifications and payments are shared, so a new kind of group buying, like event tickets, could reuse them.*

≈ 85 words · ≈ 39 s at 130 wpm · slot 40 s

**Cues**

- The future seeds are on the slide (briefing success criterion "support for more seeds in future"); no need to list them all aloud.

**Hand-over** *"Here's what we actually built."*

### Slide 4 Scope: the full buying journey, on three services

16:16:35–16:17:00 · 25 s

**Speaker** Lim · covers guideline item **1c**

**Say** *We built the whole buying journey: sign in, browse, start or join a team, pay, and then the team either succeeds, or expires and everyone is refunded. It runs on three services. We left out multiple regions, real money and email. Moving payments into a service of its own is our planned next step.*

≈ 54 words · ≈ 25 s at 130 wpm · slot 25 s

**Cues**

- 16:17:00 — questions (1 minute). Lim repeats each question in one line and names who answers.

**Hand-over** (after questions) "Now, where the project stands."

## Q&A (16:17–16:18)

| Likely question | Answer: decision → reason → evidence or limitation (30–45 s) | Owner |
|---|---|---|
| What makes this a platform rather than one application? | Two services, identity and notification, are shared: any new kind of group buying can use them as they are. A new listener is just a new queue, and a new service starts from our template with login and error handling built in. Eric covers this in segment 3. | Lim → Eric |
| Why build group buying at all? Isn't this a solved problem? | Communities still organise it in chat groups. The hard parts are keeping track, collecting money and doing refunds, and the platform does all of that automatically. | Lim |
| What would a second business on the platform need? | Its own catalogue and rules, in a new service built from our template. Sign-in, notifications and payments are reused as they are. Examples are on slide 3. | Lim |

# 2 Project Conduct (1.5+0.5 minutes)

| | |
|---|---|
| **Clock** | 16:18:00 – 16:20:00 (1.5 min talk + 0.5 min Q&A; talk ends 16:19:30) |
| **Owner** | Lim Jiechao |
| **Slides** | 5, 6 |

**Guideline requirement (V2.8)**

**2a** — Explain the current status of the project including outstanding issues.
**2b** — Describe the actual project milestones and the rough total effort expended by each team member.

## Slide 5 Status: what is verified on AWS, and what is still open

16:18:00–16:18:45 · 45 s

**Speaker** Lim · covers guideline item **2a**

**Say**

*So where are we today? On 17 October we built the full three-service setup on AWS from nothing. All nine smoke tests passed, and teardown removed all 115 resources. On 19 October we ran the auto-scaling test on AWS, and commerce scaled itself from one task to three. The other drills did not run on AWS; the middle column shows any local result. Two things are still open. An interrupted start-up can leave auto-scaling switched off; for now we fix that by hand. And the database runs in one zone, a deliberate choice to save cost.*

≈ 96 words · ≈ 44 s at 130 wpm · slot 45 s

**Cues**

- Say only what docs/swe5001/evidence/ records; the assessors may probe status claims.
- The middle column says "not run on AWS" for every drill that was only tested locally; say it the same way.

**Hand-over** *"And how we got here."*

## Slide 6 On schedule, about 50 person-days across five people

16:18:45–16:19:30 · 45 s

**Speaker** Lim · covers guideline item **2b**

**Say**

*We stayed on the plan from our proposal. We handed it in on 4 October, deployed to AWS for the first time two days later, split into three services by the 14th, measured our load baseline on the 16th, made the first clean three-service release on the 17th, and ran our test session on the 19th. In total that's about 50 person-days, in line with what the course expects. Each person's share is based on the issues they actually delivered.*

≈ 80 words · ≈ 37 s at 130 wpm · slot 45 s · if short of time: If over time, name only three dates (4, 14 and 17 October).

**Cues**

- 16:19:30 — questions (30 seconds).

**Hand-over** (after questions) "Eric will take you through the logical architecture."

**Watch out**

Slide 5 shows the real state: only auto-scaling ran on AWS (tick); the drills say “not run on AWS”, with their local result where there is one. Never claim a result the evidence folder does not record.

## Q&A (16:19:30–16:20)

| Likely question | Answer: decision → reason → evidence or limitation (30–45 s) | Owner |
|---|---|---|
| How was the work split, and is the effort within the guideline? | About 50 person-days in total, which fits the course guideline: between 8.5 and 12.5 days per person. We added up the planned hours of the issues each person actually delivered, and the team agreed the final split. | Lim |
| What slipped? | Say it plainly from slide 5, then say how and when it will be closed. The full results table is backup B12. | Lim |

# 3 Logical Architecture & Design (7+2 minutes)

| | |
|---|---|
| **Clock** | 16:20:00 – 16:29:00 (7 min talk + 2 min Q&A; talk ends 16:27:00) |
| **Owner** | Wang Ziang (Eric) |
| **Slides** | 7, 8, 9, 10, 11 |

**Guideline requirement (V2.8)**

**3a** — Highlight the key architectural decisions that influence the logical architecture.
**3b** — Explain the logical components of the solution by showing a logical detail deployment diagram and other supporting views.
**3c** — Where relevant, highlight the elements of Domain Driven Design that help to shape the design of the solution as a platform. In particular, describe how the platform is designed to interact with the producers and consumers.

**Tip**

Slide 7 is the most important slide of the talk: it is the one decision assessors will remember. Rehearse it until it fits in 90 seconds without rushing.

## Slide 7 Why three services, not five

16:20:00–16:21:30 · 90 s

**Speaker** Eric · covers guideline item **3a**

***Say***

*Thanks, Lim. I'm Eric. Our biggest decision was where to split the system. The obvious answer: one service per business area, like catalogue, orders and payments. But look at a Stripe payment. In one database transaction, we confirm the payment, create the order, take the stock and add the buyer to the team. Four steps, one commit. If any step fails, all of it is undone, and that's what keeps a buyer's money safe. Put those steps in separate services, and we'd need complicated undo logic everywhere. We estimated five services at 50 to 55 person-days, our whole budget, so we said no. So our rule: only split where no transaction crosses the line. Identity and notifications pass that test. Identity is used through one small interface, and nothing needs to call notifications. That leaves two reusable platform services, and one core service, commerce, that keeps the money flow together. Three services, not five.*

≈ 154 words · ≈ 71 s at 130 wpm · slot 90 s

**Cues**

- Pause on the left box: this is the one decision assessors should remember.
- Lim moves to the next slide as Eric starts each of slides 8–11. Eric's pointing cues for slides 7–11 and 23 are in notes/speaker-script-eric.md.

**Hand-over** *"But even this split raises three questions."*

## Slide 8 Four decisions shape the logical design

16:21:30–16:22:50 · 80 s

**Speaker** Eric · covers guideline item **3a**

**Say**

*Top left is that rule. The other three answer those three questions. How do services talk without a shared transaction? Through events. We save each event in the same transaction as the change itself, which means an event only goes out if the change really happened. We never announce a payment that was rolled back. How does each service know who you are? All services share one login session, so any service can check your role without asking identity. What if commerce can't reach identity? It refuses the purchase, because a purchase is either fully done, or not done at all. The price we pay: notifications arrive about two seconds later, and purchases pause while identity is down. We tested that outage locally; it's in our backup slides.*

≈ 128 words · ≈ 59 s at 130 wpm · slot 80 s · if short of time: If over time, name the four decisions and skip the strip at the bottom (the "price we pay" lines).

**Hand-over** *"That pause comes from our one direct call between services. Here's where it sits."*

## Slide 9 Logical view: one direct call, everything else by events

16:22:50–16:24:20 · 90 s

**Speaker** Eric · covers guideline item **3b**

**Say**

*This is the logical view of those three services. The two shared platform services are at the top, the business application, commerce, is at the bottom, and the event system sits between them. Requests enter through CloudFront and a load balancer, which picks the right service. The red arrow is that direct call. Commerce asks identity, “is this user still active?” That's the one small interface from slide 7. It's the only place where one service waits for another, so it's the only place we had to plan for that failure. Everything else goes through events, so nothing else waits. Commerce saves each event in a small outbox table, a relay sends it to an SNS topic, and each listener gets its own queue. Today: notifications and an audit queue. Each service has its own database, and they all share one session store: that's the one login session from slide 8.*

≈ 151 words · ≈ 70 s at 130 wpm · slot 90 s

**Cues**

- Walk the diagram top to bottom: platform services, the red direct call, the event bus.
- The plainer v2 diagram is backup B13 if a simpler companion is needed in Q&A.

**Hand-over** *"So why are the lines exactly there? The domain model."*

## Slide 10 Bounded contexts decide the services; events are the contract 16:24:20–16:25:35 · 75 s

**Speaker** Eric · covers guideline item **3c**

**Say** *We found those lines by following the life of a team. It's created, people join and pay, then it succeeds, or expires and gets refunded. That gave us six bounded contexts, in Domain-Driven Design terms. Then we applied the rule. Four of them, catalogue, group buying, orders and payments, share transactions, like the payment from slide 7. So they live together in commerce. Identity and notification each stand alone. Inside commerce, these areas talk through internal events, like "team expired". When an event leaves the service, it becomes a public event with a version number: "team expired, version 1". That's the contract between services. It only carries IDs, never names or emails, so personal data stays in identity. Any new service can listen to it without knowing how commerce works inside.*

≈ 131 words · ≈ 60 s at 130 wpm · slot 75 s

**Cues**

- If asked for the relationships between contexts, open backup B3 (context map).

**Hand-over** *"So who uses the platform?"*

## Slide 11 How producers and consumers use the platform, and what reuse costs 16:25:35–16:27:00 · 85 s

**Speaker** Eric · covers guideline item **3c**

**Say** *Two sides. Producers: sellers and admins post listings, and buyers start and share teams. Consumers: they browse, join, track orders and get notified. Both sides use the same account. And because whatever happens in commerce becomes an event, reuse is cheap, and we can measure it. Remember the audit queue? Two lines of configuration, and no change to commerce. A whole new service is one entry in our infrastructure template, which sets up everything it needs to run. All clients use one published API description, and our web app's client code is generated from it. And every new service starts with login, error handling, tracing and duplicate protection already built in. So, back to where I started: where to split the system? Only where no transaction crosses the line. That gave us three services, not five, two of them reusable platform services. Happy to take questions.*

≈ 146 words · ≈ 67 s at 130 wpm · slot 85 s · if short of time: If over time, drop the "every new service starts with" sentence (the strip at the bottom says it).

**Cues**

- 16:27:00 — questions (2 minutes).

**Hand-over** (after questions) "Thanks. That's the logical side; Liu Jin will show how these three services run on AWS."

### Q&A (16:27–16:29)

| Likely question | Answer: decision → reason → evidence or limitation (30–45 s) | Owner |
|---|---|---|
| Isn't commerce still a monolith? | Yes, on purpose. One payment touches the payment, the order, the stock and the team in a single transaction. Splitting that up would need undo logic everywhere, about 50 to 55 person-days. Inside, commerce is still split into clear layers (backup B5), and moving payments out is our next step (backup B2). | Eric |

| | | |
|---|---|---|
| Why not one service per business area? | Same reason: those areas share transactions. Point back to the four steps and the rejected five-service option on slide 7. The full context map is backup B3. | Eric |
| How does each service know a user's role? | When you sign in, identity writes your roles into a shared login session. The other services read that session, so they never have to ask identity on each request. A role change takes effect the next time you sign in. | Eric |
| Why do events have version numbers? | Because other services depend on them, like a public contract. A listener simply ignores versions it doesn't know, so we can add a version 2 without updating every service at the same time. | Eric |
| What did Domain-Driven Design actually change? | It gave us the candidate boundaries. Then we checked which of them share a transaction: those stayed together, and only the rest became separate services. It also made us separate internal events from public ones. Backup B4 shows the main objects and their states. | Eric |
| How would outside partners use the platform? | Through one published API description, with generated client code and a check on every pull request, plus the versioned events. Not built yet: an API gateway, keys per partner and rate limits. Backup B10. | Eric |
| Does anything read the audit queue yet? | Not yet. It's a queue that receives every team event, to show how cheap a new listener is. An audit or analytics service could read it later without any change to commerce. | Eric |

# 4 Physical Architecture & Design (7+2 minutes)

| | |
|---|---|
| **Clock** | 16:29:00 – 16:38:00 (7 min talk + 2 min Q&A; talk ends 16:36:00) |
| **Owner** | Liu Jin (4a–4c), Yu Qianshuo (Yorik) (4d) |
| **Slides** | 12, 13, 14, 15, 16, 17 |

**Guideline requirement (V2.8)**

**4a** — Highlight the key architectural decisions that influence the physical architecture.
**4b** — Describe the physical architecture that includes the network, nodes, technologies and cloud services that are used in the solution.
**4c** — Describe the persistence design of the solution.
**4d** — Describe the detailed design of an architecturally representative user case or user story.

**Tip**

Liu Jin owns slides 12–15 (about 4.3 minutes); Yorik owns 16–17 (about 2.7 minutes). The hand-over line is on slide 15; Yorik unmutes during its last sentence. Slide 16 is the UML sequence with two click reveals; if it does not feel right at the 20 Oct rehearsal, present backup B15 (numbered steps) with the same words.

## Slide 12 Physical decisions: managed, disposable, low cost

16:29:00–16:30:25 · 85 s

**Speaker** Liu Jin · covers guideline item **4a**

***Say***

*Hi, I'm Liu Jin. Let's look at how this runs on AWS. Our containers run on Fargate, on ARM processors, so there are no servers to look after, scaling comes built in, and ARM is about twenty percent cheaper. We did look at Kubernetes, but its control plane alone would cost more than all of our computing. We also don't keep the environment running: we build it when we need it and tear it down afterwards. All three services share one database server, though each has its own database. And instead of AWS's NAT gateway, we run a small NAT instance: about four dollars a month instead of forty-three. Neither our pipeline nor our services use long-lived access keys. The catch is less redundancy, which is fine for a project this size. So what does it actually cost? We checked the bill: seven full sessions in mid-October came to forty-nine cents for the database, the cache and the containers.*

≈ 159 words · ≈ 73 s at 130 wpm · slot 85 s

**Hand-over** *"Here's what that looks like."*

## Slide 13 Physical view: private services behind CloudFront, across two zones

16:30:25–16:31:50 · 85 s

**Speaker** Liu Jin · covers guideline item **4b**

**Say** *Everything runs in Singapore, spread across two availability zones. Only the load balancer and the NAT sit in public subnets; the services, the database and the cache are all private. CloudFront is the only way in. It serves the web app and passes API calls on to the load balancer, and the load balancer turns away anything that doesn't carry CloudFront's secret header. Around that, our images live in ECR, secrets in Parameter Store, events go through SNS and SQS, and CloudWatch keeps an eye on all of it. The blue boxes stay up all the time. Everything inside the red dashed box is rebuilt on every release. So security comes in layers: one way in, private services, no access keys in the pipeline, and no secrets in the code.*

≈ 130 words · ≈ 60 s at 130 wpm · slot 85 s

**Cues**

- Point at the blue "permanent" boxes, then the red "rebuilt every release" box.
- The plainer v2 diagram is backup B14.

**Hand-over** *"Each service scales on its own signal."*

## Slide 14 Each service scales on its own signal, and how far it goes

16:31:50–16:32:35 · 45 s

**Speaker** Liu Jin · covers guideline item **4b**

**Say** *Commerce adds tasks as requests per task go up, from two to six. Notification follows its queue, from one to four. On the sixteenth, one task handled 200 requests a second with room to spare. On the nineteenth, with all three services up, commerce grew from one task to three on its own, serving about 455 requests a second, 95 percent in under 50 milliseconds. That was all our laptop could send. Six tasks should manage around 1,200 a second, but that's an estimate. Beyond that, the single database is the limit.*

≈ 92 words · ≈ 42 s at 130 wpm · slot 45 s · if short of time: If over time, skip the chart; read only the capacity line.

**Cues**

- Never present the projection as a measurement. 455 req/s was the laptop's limit, not the service's: if asked, say so.

**Hand-over** *"Now the data."*

## Slide 15 Each service owns its data

16:32:35–16:33:20 · 45 s

**Speaker** Liu Jin · covers guideline item **4c**

**Say** *Each service owns its data. They share one database server, but each has its own database, and permissions stop a service from reading anyone else's tables. The cache holds sessions and short-lived data; we never store card details. One tricky case: two buyers going for the last seat at the same time. We always lock in the same order, team, then purchase, then payment, so exactly one of them wins. Scheduled jobs take a database lock too, so only one copy runs. Yorik will now walk you through one use case.*

≈ 91 words · ≈ 42 s at 130 wpm · slot 45 s

**Hand-over** *"Yorik, over to you."*

## Slide 16 Team expiry, end to end: one transaction in, one transaction out

16:33:20–16:34:55 · 95 s

**Speaker** Yorik · covers guideline item **4d**

**Say** *I'm Yorik. This is what happens when a team runs out of time. Every minute, a scheduled job looks for expired teams, and a database lock makes sure only one copy of it runs. In one transaction, it marks the team as expired, cancels the orders, records the refunds that are owed, and saves a "team expired" event. Either all of that happens, or none of it does.* **[click]** *Once that's committed, the relay sends the event to SNS, which puts it on the notification queue. The notification service picks it up, checks the event ID so a duplicate is ignored, and creates a notification for each member, again in one transaction. Only then is the message deleted from the queue.* **[click]** *Refunds run on their own. Every thirty seconds, a refund worker takes the next refund and sends it to Stripe with a unique key, so even if it retries, nobody is refunded twice. So: one transaction in commerce, one in notification, a queue in between, and refunds that are safe to retry.*

≈ 172 words · ≈ 79 s at 130 wpm · slot 95 s

**Cues**

- Two clicks: the slide opens on the expiry transaction (step 1); click 1 shows relay → SNS → SQS → notification (steps 2–8); click 2 shows the refund worker loop.
- Fallback if the UML version does not feel right in rehearsal: backup B15 (numbered steps) with the same words.

**Hand-over** *"And when things go wrong?"*

## Slide 17 Failures are expected, and handled

16:34:55–16:36:00 · 65 s

**Speaker** Yorik · covers guideline item **4d**

***Say*** *A queue can deliver the same message more than once, so we designed for that. If a message arrives twice, the second copy is ignored, because we've already recorded its ID. If a message keeps failing, after five tries it's moved aside into a dead-letter queue and an alarm goes off, while everything else keeps flowing. We tested this locally: a broken message was tried five times, then moved aside. And if sending an event fails, it simply stays in the outbox and is tried again. So no event is lost, nobody gets the same notification twice, and one bad message can't block the rest.*

≈ 105 words · ≈ 48 s at 130 wpm · slot 65 s

**Cues**

- 16:36:00 — questions (2 minutes). Liu Jin answers infrastructure and data; Yorik answers events and failures.

**Hand-over** (after questions) "Pan will show how we build and ship it."

### Q&A (16:36–16:38)

| Likely question | Answer: decision → reason → evidence or limitation (30–45 s) | Owner |
|---|---|---|
| What breaks first at higher scale? | The single database server, then the relay that sends events, at about 50 events a second. That number is calculated, not measured. The fixes: give busy databases their own server or read copies, and run several relays side by side. | Liu Jin |
| Aren't one database and one cache single points of failure? | Yes, and that was a cost choice for this project. The services already run across two zones. For production we'd add a standby database and a cache replica (backup B2, AISS-01). | Liu Jin |
| Why Fargate and not Kubernetes? | Kubernetes on AWS charges about 73 dollars a month just for its control plane, more than all our computing. Fargate scales on its own and has no servers to manage, and three services don't need Kubernetes. Backup B1b. | Liu Jin |
| How are the internal endpoints protected? | The load balancer doesn't route them, only our own services can reach them, and they're not in the public API; a test checks that. There's no service token yet. We accepted that limit (AISS-04). | Liu Jin |
| Where does the 12,000 requests per minute target come from? | Our 16 October load test: one task handled 200 requests a second with fast responses and no errors, and it still had room. 200 a second is 12,000 a minute. On 19 October, with all three services, that target scaled commerce from one task to three. | Liu Jin |
| Slide 14 says two to six, so why did your test start at one? | Two is the floor we normally run with, one task in each availability zone. But our per-task figure, 200 requests a second, was measured on a single task, so the scaling test starts from one to get a clean baseline. The test tool even checks that before it sends any traffic. We lowered the floor to one for the test and put it back to two afterwards. (Do not claim the two tasks were verified in two zones after 19 October: that was not checked.) | Liu Jin |
| How do you make sure events are never lost or doubled? | Each event is saved in the same transaction as the change, so it can't be lost. A queue may deliver it twice, so each listener records the event ID together with its own change, and a repeat does nothing. | Yorik |

| | | |
|---|---|---|
| What happens when identity is down? | Browsing keeps working. Starting a purchase or an admin change fails safely with a server error, so nothing half-done is saved. For a Stripe payment, our webhook returns an error, nothing is committed, and Stripe simply tries again later. Once identity is back the payment goes through, and a repeated retry changes nothing. We tested this locally, not on AWS; the results are in backup B12. The limit: checkout needs identity to be up (AISS-03). | Yorik |
| Why SNS and SQS, and not Kafka or RabbitMQ? | They're fully managed and you pay only for what you use, with nothing to run. SNS copies each event to every queue that wants it, and the queue length drives scaling and gives us dead-letter queues. Kafka would be far too much for our volume. | Yorik |
| Can you show the payment path instead of expiry? | Yes, backup B6: the signature is checked first, then one transaction checks for duplicates, locks the records, adds the member, saves the order, takes the stock and saves the event. | Yorik |

## Liu Jin: extended Q&A bank (slides 12–15)

> **Tip**
> Rows above stay the first answers. These are the likely follow-ups, checked against the repository on 20 Oct. Answers in brackets are notes for Liu Jin, not to be read out.

### Slide 12: physical decisions

| Likely question | Suggested answer (say it in 30–45 s) | Who answers |
|---|---|---|
| Why ARM? Did that cause problems? | ARM Fargate is about 20 percent cheaper for the same size. Java 21 runs natively on ARM, and CI builds every image for arm64, so there was nothing to port. | Liu Jin |
| You use Fargate Spot. What happens when AWS takes a task back? | The minimum tasks always run on regular Fargate; only extra tasks use Spot, weighted three to one. If a Spot task is reclaimed, ECS starts a replacement and the load balancer drains the old one. A failed deployment rolls back by itself through the circuit breaker. | Liu Jin |
| How did you measure 49 cents? | From AWS Cost Explorer, filtered to the runtime services (database, cache and containers) for 15 to 17 October, seven sessions. It excludes the permanent edge and the development server. While up, the environment costs about 12 to 20 cents an hour. (No evidence file in the repository: keep a Cost Explorer screenshot ready.) | Liu Jin |
| Isn't one NAT instance a single point of failure? | Yes, for outbound traffic. It's a small fck-nat instance in an auto-scaling group of one: if it dies, a replacement takes over the same network interface, so no routes change. During that gap, calls to AWS APIs pause; events stay in the outbox and are retried, so nothing is lost. In production we'd use NAT gateways or VPC endpoints. (Image layers use the S3 gateway endpoint; ECR API, SSM, SNS, SQS and CloudWatch go through the NAT.) | Liu Jin |
| You destroy the environment every time. What about the data? | It's a disposable test environment: the database is rebuilt each session, with no backups, and the demo data is reseeded automatically. Production would turn on backups, point-in-time recovery, Multi-AZ and deletion protection. | Liu Jin |
| How long does a rebuild take? | About 26 to 32 minutes, of which about 15 are spent waiting for the SNS topic (AISS-06). On the day we start early. | Liu Jin |

## Slide 13: physical view

| Likely question | Suggested answer (say it in 30–45 s) | Who answers |
|---|---|---|
| Is traffic encrypted end to end? | Users reach CloudFront over HTTPS, and services talk to the database and the cache over TLS; the database enforces it, and the cache also needs a password. The hop from CloudFront to the load balancer is plain HTTP today, locked down by CloudFront's IP ranges and a secret header; moving it to HTTPS with a certificate is a known next step. (Do not claim that hop is encrypted.) | Liu Jin |
| What if someone finds the load balancer's address? | Its security group only accepts CloudFront's IP ranges, and even from CloudFront a request without the secret header gets a 403. | Liu Jin |
| Do you use a WAF? | Not yet. The next step is AWS WAF on CloudFront, with managed rules and rate limits. | Liu Jin |
| Where are the secrets? | In Parameter Store, encrypted. Database passwords are generated as write-only values, so they never land in Terraform state, and nothing is in the code. (The cache's auth token is in Terraform state: do not say "no secrets in state".) | Liu Jin |

## Slide 14: scaling

| Likely question | Suggested answer (say it in 30–45 s) | Who answers |
|---|---|---|
| Why scale on requests and not CPU? | Browsing is driven by requests, and we measured capacity per task in requests. CPU reacts later and is noisier. Commerce scales out after 60 seconds and in after 180 seconds, so it doesn't flap. | Liu Jin |
| How does notification scale? | On queue backlog: about 50 waiting messages per task, between one and four tasks. Identity stays at one task. | Liu Jin |
| How many database connections do six commerce tasks use? | Each task has a pool of ten connections. At full scale, six commerce, four notification and one identity task, that's up to 110, close to what a micro database allows. On 19 October the peak was 50 connections with the database under 19 percent CPU. Past that we'd shrink the pools, move to a bigger instance or add RDS Proxy. (Before the talk, read the real limit with infra/swe5001/scripts/connection-budget.sh on the live environment; the budget was not checked formally before V3.) | Liu Jin |

## Slide 15: data

| Likely question | Suggested answer (say it in 30–45 s) | Who answers |
|---|---|---|
| Why one database server and not one per service? | Cost. Each service still has its own database and its own roles, and moving one to its own server needs no code change (AISS-05). | Liu Jin |
| How do you stop commerce reading identity's data? | Identity's tables are in a separate database that commerce isn't allowed to connect to. Each database has an owner role and an app role, and the permissions are re-applied every time a service starts. | Liu Jin |
| What exactly is the lock order? | Team first, then the purchase, then the payment, each with SELECT … FOR UPDATE. Because every code path locks in the same order, two buyers can't deadlock: the first to lock the team gets the last seat, and the second sees the team is full. | Liu Jin |

| | | |
|---|---|---|
| How does the scheduled-job lock work? | Each job holds a short lease in a database table: two minutes, renewed every third of that. If another task holds the lease, it skips the run; if renewal fails, the batch stops. Only commerce runs scheduled jobs. | Liu Jin |
| Why the database for that lock, not Redis? | The database is already the source of truth for these jobs, so the lock lives next to the data it protects and needs no extra dependency. | Liu Jin |
| Do you store card data? | No. Payment uses Stripe’s hosted checkout, so card numbers never reach our servers; we keep only Stripe’s payment references. | Liu Jin |

# 5 DevOps and Development Lifecycle (6+1 minutes)

| | |
|---|---|
| **Clock** | 16:38:00 – 16:45:00 (6 min talk + 1 min Q&A; talk ends 16:44:00) |
| **Owner** | Panpakorn Siripanich (Pan) |
| **Slides** | 18, 19, 20, 21, 22 |

**Guideline requirement (V2.8)**

**5a** — Describe the strategy to manage the source code and artifacts.
**5b** — Describe the DevOps pipeline and procedures.
**5c** — Highlight any technical findings and issues encountered.

**Tip**

Segment 5 now has five slides in the same 6 minutes. The hand-over to the demo moved from slide 21 to slide 22.

## Slide 18 One main branch; production changes only by promotion

16:38:00–16:39:05 · 65 s

**Speaker** Pan · covers guideline item **5a**

**Say**

*I'm Pan. All work happens on short-lived branches that merge into our main branch, dev, after a review and automated checks. Production lives on a separate branch, prod. The only way to change it is a promotion pull request from dev; nobody can push to it directly. For build outputs, we store each container image by its unique digest rather than by a name tag, keep a record of which images are live, and keep one API description file in the repository.*

≈ 82 words · ≈ 38 s at 130 wpm · slot 65 s

**Hand-over** *"Every change is checked before it merges."*

## Slide 19 Every pull request is checked before it can merge

16:39:05–16:40:15 · 70 s

**Speaker** Pan · covers guideline item **5b**

**Say** *Every pull request goes through four groups of checks. First, build and test: unit tests, an API check, and at least 80 percent test coverage on new code. Second, containers: we build and scan every service image, and start the app to make sure it's healthy. Third, security: we scan for leaked secrets, check our libraries and images for known weaknesses, and run code analysis in SonarCloud. Fourth, infrastructure: we check and test our Terraform code and run policy checks. Over the past month that came to more than five thousand runs, about 95 percent passing, usually in under nine minutes. If any check fails, the change can't be merged.*

≈ 110 words · ≈ 51 s at 130 wpm · slot 70 s · if short of time: If over time, one sentence for all four groups.

**Hand-over** *"And this is how a change reaches production."*

## Slide 20 One promotion builds, deploys, tests and releases, then destroys it all

16:40:15–16:41:50 · 95 s

**Speaker** Pan · covers guideline item **5b**

**Say** *This is the key part. The only thing that deploys to production is merging a promotion pull request. The pipeline then builds the images, creates the whole AWS environment from scratch, deploys the services and runs smoke tests through CloudFront. If a release fails its health checks, it rolls itself back. Then the pipeline marks those images as the new release, destroys everything, and checks that nothing was left behind. That last step always runs, even if something failed earlier. Because every release starts from nothing, every release is also a disaster-recovery test. Our first run on 6 October, with one service, took 28 minutes. The three-service run on 17 October took 51. About half an hour of that is creating the environment, and fifteen minutes of it is just waiting for new SNS subscriptions to start working. Which brings me to what we learned.*

≈ 145 words · ≈ 67 s at 130 wpm · slot 95 s

**Cues**

- Timings from run 39008208244 (17 Oct): build 3 min + publish 2 min, create + deploy + smoke 32 min, release 1 min, destroy 13 min.

**Hand-over** *"Three lessons we learned on the way."*

## Slide 21 What the real cloud taught us

16:41:50–16:43:05 · 75 s

**Speaker** Pan · covers guideline item **5c**

**Say** *The real cloud taught us three things. First, our infrastructure tests passed with mock-ups, but the first real deployment failed because of missing permissions. Mocks only test what you tell them, so now we test permissions for real. Second, a new SNS subscription can take up to fifteen minutes to start delivering. Nothing shows an error; the messages just disappear. So our deploy now waits until delivery is confirmed. Third, a start-up that was interrupted and retried said it had succeeded, but left auto-scaling switched off. Deployed doesn't mean scaling, so now we check.*

≈ 94 words · ≈ 43 s at 130 wpm · slot 75 s

**Cues**

- The full findings list is backup B8.

**Hand-over** *"And who and what can touch AWS."*

## Slide 22 No standing keys: who and what can touch AWS

16:43:05–16:44:00 · 55 s

**Speaker** Pan · covers guideline item **5b**

**Say** *Finally, who and what can touch our AWS account. Security starts above the account itself. It sits in an AWS Organization whose rules keep us in Singapore and block expensive services, and those rules apply even to our own admins. People sign in through Identity Center, with sessions that end after an hour. The pipeline gets short-lived access from GitHub, with separate roles to plan, deploy and apply, so no AWS keys are stored in GitHub. There's one exception, and we're open about it: two IAM users for database access, still under the same rules. Eric will now show the system live.*

≈ 102 words · ≈ 47 s at 130 wpm · slot 55 s

**Cues**

- 16:44:00 — questions (1 minute).
- 16:44:30 — Lim stops sharing; Eric starts sharing the demo window.

**Hand-over** (after questions) "Eric, over to you."

**Watch out**

At 16:44:30 Lim stops sharing and Eric starts sharing the demo window, even if a question is still being answered. The answer continues over Eric's screen.

### Q&A (16:44–16:45)

| Likely question | Answer: decision → reason → evidence or limitation (30–45 s) | Owner |
|---|---|---|
| How do you stop one change from breaking a service? | Every pull request builds and scans every service image and starts the app to check it's healthy. On top of that, 18 automated architecture rules stop code from crossing service boundaries (backup B5). | Pan |
| Why destroy the environment after every release? | Cost, and proof. Running only when needed costs about 20 cents an hour instead of about 125 dollars a month; seven real sessions cost 49 cents. And because every release builds everything from nothing, we know we can rebuild at any time. Backup B9. | Pan |

| | | |
|---|---|---|
| What security checks are in the pipeline? | Scans for leaked secrets, for known weaknesses in our libraries and images, and for risky infrastructure settings, plus code analysis in SonarCloud. Deploy roles only work for the prod branch, and secrets live in Parameter Store, never in code. The account-level rules are on slide 22. | Pan |
| How do you roll back? | If a new version fails its health checks, ECS rolls back to the last good one by itself. We also keep a record of the last good images, so we can always rebuild from them. | Pan |
| Why are there still IAM users? | Two users for database access, with one active key, shown openly on slide 22. They're still under the organisation's rules. Everyone and everything else signs in through Identity Center or short-lived GitHub access. | Pan → Lim |

# 6 Demo (8+2 minutes)

| | |
|---|---|
| **Clock** | 16:45:00 – 16:55:00 (8 min talk + 2 min Q&A; talk ends 16:53:00) |
| **Owner** | Eric (drives), Pan (pipeline) |
| **Slides** | 23 |

**Guideline requirement (V2.8)**

**6a** — Demonstrate some of the significant use cases of the solution.
**6b** — Demonstrate how the solution deals with some of the important quality attributes e.g., scalability (such as demonstrating auto-scaling), performance (show using some scripted performance testing), maintainability (show/run the DevOps pipeline).

## How the demo meets 6a and 6b

| Step | Shows | Quality attribute (guideline 6b wording) | Evidence on screen |
|---|---|---|---|
| 1 | Buy as a customer, across three services | Functionality | All four views load; at least one notification is listed. |
| 2 | Auto-scaling, on the dashboard | Scalability, performance | Live dashboard loads with commerce at 2 running tasks; the 19 Oct graphs show the task count going 1 → 2 → 3 and server p95 under 50 ms. |
| 3 | The pipeline that deployed it | Maintainability | Checks green; run green including destroy; artifact present. |
| 4 | Close the demo | Reliability (tested locally) | Nothing to operate. 16:51:30–16:53 is spare: questions on the demo. |

Scalability and performance are shown in step 2: the live dashboard, then the CloudWatch graphs of the 19 October k6 run on AWS. Maintainability is shown through the pipeline in step 3. Three drills were cut from the live demo on 19 October because they were not rehearsed on AWS: stopping a server, the identity outage and the event backlog with the dead-letter queue. Slide 23 says so in one line, and slide 5 and backup B12 show their results. The demo now ends at about 16:51:30; the spare time goes to questions.

## Principle: nothing changes on the environment during the talk

The demo only shows results. No load test runs during the talk: a live V3 run needs the commerce floor lowered to one task and a fresh preflight a few minutes before the load (A.7), which is too much to operate on air (team decision, 20 Oct). Step 2 therefore shows the live dashboard, with commerce at its standing two tasks and little traffic, and then the graphs from the 19 October run on AWS. In that run the reviewed ramp took commerce from 1 to 2 tasks at 15:55 and from 2 to 3 at 15:58, all by itself, and at about 455 RPS the server answered 95% of requests in under 50 ms. The laptop sending the load hit its limit first, so 455 RPS is not the service's ceiling. Say plainly that the graphs are from 19 October; never present them as live.

## Demo preconditions (checked at 15:30 by Liu Jin and Eric)

- Environment up (the #564 promotion on 19 Oct took it down, so it is brought up again before the day), health `UP`, smoke green; commerce 2/2 (its standing floor), identity 1/1, notification 1/1 running.
- Demo admin granted and signed in again; startable listings seeded (A.5).
- Scaling targets **not suspended**: commerce 2–6, notification 1–4 (read back; see finding 3 on slide 21).
- Notifications JSON for the demo buyer shows at least one entry (from the smoke run or a seeded expiry).
- The 19 Oct evidence page (tab 6) is open with every graph loaded, and the same PNGs are ready locally in `fallback/2-v3`.
- Fallback clips for steps 1 and 3 (recorded before the talk) open in a player on Liu Jin's machine, tested with Zoom “share sound” off. No recordings yet: any step that fails live uses Plan C (§7) until they are recorded.

## Screen layout (Eric's demo window)

One browser window, tabs left to right. Share the window, not the desktop; notifications off; browser zoom 125%.

| Tab | Content | Prepared state |
|---|---|---|
| 1 | HTML deck opened at slide 23 (`swe5001-presentation.html#23`) | Also used for slide 24 at the end; the backups are one key away (B) |
| 2 | SPA `https://d3lvq02o2e7tcn.cloudfront.net` | Product list loaded; signed in as the demo admin (Google) |
| 3 | Admin product list in the SPA | Admin listing visible |
| 4 | `/api/v1/notifications` for the demo buyer | JSON list with at least one notification |
| 5 | CloudWatch dashboard `blocbuy-swe5001` | 1-hour window, auto-refresh 10 s; Edge, Services, Messaging rows visible |
| 6 | GitHub: the 19 Oct evidence page `docs/swe5001/evidence/20261019-v3-autoscaling/README.md` | Scrolled to the task-count graph; the local PNGs in `fallback/2-v3` are the backup |
| 7 | GitHub: promotion pull request #564 (19 Oct) (`dev` → `prod`) and its `swe5001-deploy` run | Checks tab and run graph open; never the Merge button |

## Opening (slide 23)

**Slide 23 Live demo** 16:45:00–16:45:15 · 15 s

| | |
|---|---|
| **Speaker** | Eric · covers guideline item **6a** |
| ***Say*** | *Thanks, Pan. Three things: the buying journey, how the system scaled under load, and the pipeline that deployed it.*<br>≈ 19 words · ≈ 9 s at 130 wpm · slot 15 s |
| **Cues** | • Eric's tab 1 shows this slide (swe5001-presentation.html#23). Follow the demo run book step by step. |
| **Hand-over** | *"First, the use cases."* |

The slide's footer says the scaling graphs are from the 19 October load test, so nobody mistakes them for live data.

## Step by step

**Step 1 Buy as a customer, across three services** 16:45:15 – 16:46:45

| | |
|---|---|
| **Proves** | Functionality · guideline **6a** · Eric |
| **Operate** | 1. Tab 2: product list with team prices and countdowns (commerce).<br>2. Point at the signed-in user in the header (identity, Google sign-in).<br>3. Tab 3: admin product list (commerce reads the role from the shared session).<br>4. Tab 4: refresh the buyer's notifications (notification service). |
| ***Say*** | *This is what a buyer sees: team prices and live countdowns, served by commerce. Up here, I'm signed in with Google, through the identity service. And because of that one login, this admin page works too: commerce reads my role from the shared login session, without calling identity. That's the one login session from slide 8. Last, these notifications. Remember, nothing needs to call notifications: the notification service created these from events it received. So that's three services, not five, behind one login. Now, those product pages come from commerce, and browsing them is the traffic commerce has to scale for. Let's see how it copes.* |
| **Expect** | All four views load; at least one notification is listed. |
| **Fallback** | Screenshots taken before the talk (fallback/1-use-cases). If the live view is not as expected within 20 seconds, Eric posts `CLIP 1` and says: *"This one's not loading in time, so here's the same step from our recording instead. What you're seeing now is recorded, not live."* With no recording for this step, use Plan C (§7). |
| **If** | **No notifications for the demo account (tab 4):** Say: "This account has no notifications yet, so here's the same view from our recording." Then post CLIP 1. |
| **If** | **Product images missing (striped tiles on tab 2):** Say: "The product images aren't loading on this environment, so you'll see placeholder tiles, but the team prices and live countdowns are what commerce serves, and that's the part to look at." |
| **If** | **Running late:** Show only the product list (tab 2) and the notifications (tab 4). |

## Step 2 Auto-scaling, on the dashboard

16:46:45 – 16:49:15

**Proves** Scalability, performance · guideline **6b** · Eric

**Operate**

1. Tab 5: live CloudWatch dashboard. Services row: commerce at 2 running tasks, its normal floor; Edge row: today's traffic.
2. Tab 6: the 19 October run on AWS (evidence page): commerce task count 1 → 2 → 3, then load balancer traffic and response time.

***Say*** *This is the live dashboard for the environment you've just used. It's quiet right now, so commerce is sitting at its usual two tasks. Rather than run a load test in the middle of our talk, here's the one we ran on AWS on 19 October. These graphs are saved from that run, not live. As the traffic ramped up, commerce went from one task to two, then to three, all on its own. And it stayed fast: even at about 455 requests a second, 95 percent of responses came back in under 50 milliseconds. That was the limit of the laptop sending the load, not of the service. So this is what Liu Jin showed on slide 14: commerce adds tasks as requests per task rise, by itself. But how did that commerce code get onto AWS in the first place? That's the pipeline. Pan?*

**Expect** Live dashboard loads with commerce at 2 running tasks; the 19 Oct graphs show the task count going 1 → 2 → 3 and server p95 under 50 ms.

**Fallback** The same 19 Oct graphs as local files (fallback/2-v3). If the live view is not as expected within 20 seconds, Eric opens them himself and says: *"The live dashboard isn't loading in time, so let's go straight to the saved graphs from our 19 October run. These are recorded, not live."*

## Step 3 The pipeline that deployed it

16:49:15 – 16:50:45

**Proves** Maintainability · guideline **6b** · Pan narrates; Eric clicks

**Operate**

1. Tab 7: promotion pull request #564 (19 Oct): required checks and approval.
2. Open its deploy run: build → create → release → destroy → evidence, with durations.
3. Open the evidence artifact list. Never click Merge.

***Say*** *Pan: Thanks, Eric. Remember, the only thing that deploys to production is merging a promotion pull request. This is the promotion that put today's code on AWS. It had to pass its required checks and get an approval before it could merge. Eric, could you open the deploy run? (Eric opens it.) So this is what that merge started. The run built the images, created the environment from scratch, tested it, released it and tore it down again. End to end, it took (read the total run time off the screen). And the evidence, please. (Eric opens the artifact list.) Here's the evidence it kept. So today's environment, the one you've just used, was built from exactly these images. Eric, back to you.*

**Expect** Checks green; run green including destroy; artifact present.

**Fallback** Screenshots of the pull request and run (fallback/3-pipeline); Pan carries on with the same narration over them. If the live view is not as expected within 20 seconds, Eric posts `CLIP 3` and says: *"GitHub isn't cooperating, so here are the same pages from our recording. This part is recorded, not live."* With no recording, Eric shows those screenshots; never Plan C for this step.

## Step 4 Close the demo

16:50:45 – 16:51:30

**Proves** Reliability (tested locally) · guideline **6b** · Eric, then Lim

**Operate**

1. Tab 1: back to the deck; open backup B12 if asked about failures.
2. Thank-you slide (slide 24) at 16:53.

***Say*** *What we didn't show live is failure handling. The identity outage from slide 8 and a bad message going to the dead-letter queue were tested locally, and the results are in our backup slides; stopping a server is still to run on AWS. So: the buying journey, auto-scaling and the pipeline. That's our demo.*

**Expect** Nothing to operate. 16:51:30–16:53 is spare: questions on the demo.

**Fallback** —

### Watch out

**Never live:** clicking Merge on any PR into `prod` (a promotion destroys this environment); opening Parameter Store values, Terraform state or plan output; scaling commerce or notification by hand (autoscaling owns them); stopping tasks or services (the drills are not part of this demo).

### Tip

Narrate results, not clicks: "task count went from one to three", not "now I'm clicking on the services row". No troubleshooting on air.

# Closing slide

**Slide 24 Thank you** 16:53 · Q&A

**Speaker** Lim · covers guideline item **6b**

**Say** *Thank you. We're happy to take your questions.*

≈ 8 words · ≈ 4 s at 130 wpm

**Cues**

- 16:55:00 — end. Lim thanks the assessors. Liu Jin tears the environment down only after the meeting ends.
- Backup slides follow (press B for the index). Open one only when a question needs it.

## Q&A (16:53–16:55)

| Likely question | Answer: decision → reason → evidence or limitation (30–45 s) | Owner |
|---|---|---|
| How did you choose the scaling target? | From the 16 October load test: responses stayed fast and errors stayed under one percent all the way to 200 requests a second per task. That gives 12,000 a minute. On 19 October, with all three services, that target scaled commerce from one task to three. | Eric |
| Does adding tasks scale in a straight line? | Not proven yet. On 19 October one task topped out near 280 requests a second and three tasks served about 455, but the laptop sending the load hit its own limit first, so we can’t claim a straight line. Backup B12. | Eric |
| What analytics does the platform have? | Operational ones are in use: the dashboard, the alarms, and the load results that set our scaling target. Business ones, like how fast teams fill or how many referrals come from invites, are possible from data we already store, but not built yet. The next step is a reporting service that listens to our events. Backup B11. | Lim |
| Can you show … again? | The demo screen stays up. Eric drives, Pan shows the pipeline. Failure drills are not shown live: open backup B12. | Lim routes |
| Questions left over from earlier segments | That segment’s owner answers in 30–45 seconds, and opens the matching backup slide if it helps. | Lim routes |

# 7 Contingencies and time recovery

## If something fails

| Situation | Detected by | Response and words | Who |
|---|---|---|---|
| Environment down or unhealthy on the day | Morning check | Run `up` on `prod` (about 32 min), deadline 12:00; if it fails, `down` and retry once. If it fails again, present the demo from the recordings and the 19 Oct graphs and say so. | Liu Jin |
| Presenter's audio or connection drops | Silence > 5 s | Backup continues from the slide card: Lim ↔ Pan, Eric ↔ Yorik, Liu Jin ↔ Yorik. | Backup |
| Lim cannot share | Black screen | Eric shares the HTML deck from tab 1 for segments 1–5 (PDF as a last resort). | Eric |
| Live dashboard slow or empty | Tab 5 | Go straight to tab 6 (the 19 Oct graphs); say the live view only shows today's quiet baseline. | Eric |
| GitHub slow or logged out | Tabs 6–7 | Show the local graph PNGs (`fallback/2-v3`) and the pipeline screenshots. | Eric, Pan |
| **Plan C:** step 1 or 2 fails live and its fallback (§6 card) fails too | — | Not for step 3: its screenshots (`fallback/3-pipeline`) are the fallback, and a local test cannot show checks, a deploy run or artifacts. Show the local integration-test result for that step and say: *"verified locally, not yet on AWS."* Never present a local result as an AWS one. | Eric |
| Assessor asks for something not prepared | — | Offer to show it after the talk or in the report; do not improvise on the live environment. If a backup slide covers it, open it (B, then the row). | Lim |

## If the clock slips

Recover time in this order; never cut a guideline item, only depth.

| Segment | Cut first | Saves |
|---|---|---|
| 1 | Slide 3: skip the hand-picked offers sentence; slide 4: skip the last sentence (next step) | 15 s |
| 2 | Slide 6: name only three milestone dates | 15 s |
| 3 | Slide 8: name the decisions, skip the cost sentence; slide 11: drop the last sentence (the strip says it) | 35 s |
| 4 | Slide 12: skip the Kubernetes sentence; slide 14: skip the chart; slide 15: skip the cache and card sentence | 45 s |
| 5 | Slide 19: one sentence for all four groups; slide 21: two findings instead of three; slide 22: skip the disclosed exception (it is on the slide) | 50 s |
| 6 | Step 1: show the product list and notifications only; the demo already ends 1.5 min early | 30 s |

# 8 Timeline before, on and after the day

## Before the day

| Date | Do | Owner |
|---|---|---|
| Sun 18 Oct | Each owner reviews their slides and this script and confirms their effort figure on slide 6; send changes to Lim. | All; Lim collects |
| Mon 19 Oct | Done: V3 auto-scaling ran on AWS (evidence in PR #555); the other drills stay local and were cut from the live demo. Decision: keep the environment up until the talk; no promotion to `prod` before it (a promotion destroys the environment). | All; Liu Jin operates |
| Mon 19 Oct evening | Done: 19 Oct results filled in; deck, PDF and this script rebuilt with no gaps left. | Lim, with segment owners |
| Tue 20 Oct | Decision: no load test during the talk; demo step 2 shows the live dashboard and the 19 Oct graphs. Record the fallback clips for demo steps 1 and 3 on the running environment (1080p, no secrets visible) into `fallback/1-use-cases` and `fallback/3-pipeline`. **Full rehearsal** in Zoom, timed against the run sheet, with both share switches. Confirm slide 16 (UML) or its fallback B15. Every speaker answers their Q&A rows in ≤ 45 s. | All; Lim times |

## Presentation day (Wed 21 Oct)

| Time (SGT) | Do | Owner |
|---|---|---|
| ≤ 10:30 | A.1: nothing queued that could destroy the session. Check the environment with A.2's health and scaling commands, and run `up` on `prod` if it is down (about 32 min; the 19 Oct promotion took it down). | Liu Jin |
| ≤ 12:00 | SPA loads, health `UP`, smoke green; scaling targets read back as not suspended. Sign in with the demo Google account, A.5 grant-admin, sign out and in, seed listings. If up failed: cause → `down` → retry once. | Liu Jin |
| 13:30 | Tech check in a test Zoom: mic, camera, share window, notifications off, wired network if possible. | All |
| 15:30 | Open the demo tabs; run through the §6 preconditions. | Liu Jin, Eric |
| 16:00 | Join the meeting as "Team 8 – Name"; backchannel open; fallback clips loaded. | All |
| 16:15 | Start; follow the run sheet. | Lim |
| 16:44:30 | Share switch Lim → Eric. | Lim, Eric |
| 16:55 | End; thank the assessors. | Lim |
| ≤ 17:30 | Export the dashboard screenshot to the evidence folder. A.9: `down` and a clean `status`. Write every assessor remark into #461. | Liu Jin, Lim |

## After the presentation

- Teardown verified (A.9); record both run links in the day log. Save evidence **before** teardown: log groups die with `runtime`.

- Every assessor remark goes into #461 the same evening; each becomes a change or a justified non-change in the report (due 16 Nov).
- Tag `swe5001-demo-20261021` on the `prod` commit that was shown (never a `v*` tag).

# 9 Checks and open items before the rehearsal

## Checked against the repository (closed)

| # | Item | Finding and source | Slide |
|---|---|---|---|
| 1 | Audit consumer: lines and files | A queue plus a filter: 2 lines in infra/swe5001/modules/messaging/main.tf (PR #487); no service reads it yet | 9, 11 |
| 2 | Expiry notification: owner or each member | Each active member (IntegrationOutboxWriter.onTeamExpired → findActiveMemberIdsByDealId) | 16, B15 |
| 3 | Paused-scaling defect: fix PR open? | No fix PR; repaired by hand | 5, B8 |
| 4 | Boot check and CodeQL in CI | Service images built and scanned (swe5001-service-images.yml); backend app booted with a health check (ci-backend.yml); CodeQL off (ENABLE_CODEQL=false) | 19 |
| 5 | Relay ceiling of about 50 events/s | Calculated: 100 rows per 2-second tick (ADR 0007), not measured | 14, Q&A |
| 6 | 17 Oct run 39008208244 durations | Build 3 + publish 2, up 32, release 1, down 13 min; 51 min end to end; gaps of seconds | 20 |
| 9 | Q&A bank: single-zone data is AISS-01 | Corrected in this script | Q&A |
| 10 | Referral data for analytics | Stored: biz_share_invite, biz_share_credit, biz_purchase_intent.sharer_user_id | B11 |
| 11, 12 | Slide 22, B10, B11 graphics; recoloured UML diagrams | Built in v3 (diagrams taken from the team's v3 PDF) | 9, 13, 16, 22, B3–B6 |
| 14 | Slide 10 option A or B | Option A; the context map is backup B3 | 10 |
| 7 | Effort split (slide 6) | Agreed by all five on 19 Oct | 6 |
| 16 | Demo steps 3–5 (stop a server, identity outage, event backlog) | Cut on 19 Oct: not rehearsed on AWS; slide 23 says the identity outage and bad message were tested locally and stopping a server was not run | 23 |
| 17 | Live k6 load during the talk | Cut on 20 Oct: a live V3 run needs the commerce floor at 1 and a fresh preflight minutes before the load. Step 2 shows the live dashboard and the 19 Oct graphs; the slide 23 footer says so | 23 |
| 8 | 19 Oct results | V3 auto-scaling ran on AWS: commerce 1 → 3 tasks, about 455 RPS at server p95 under 50 ms; not V3 acceptance (generator drops). v4, v5, v7, identity, e2e shown as "not run on AWS". Evidence: docs/swe5001/evidence/20261019-v3-autoscaling/ (PR #555) | 5, 14, 24, B12 |
| 15 | Rebuild after 19 Oct | HTML, slide PDF and this script rebuilt with final: true | all |

## Still open

| # | Item | Owner | Blocks slide |
|---|---|---|---|
| — | Record the fallback clips for demo steps 1 and 3, then set recorded in the results block and rebuild | Liu Jin | 23 |
| — | Review and merge the V3 evidence pull request #555 | Eric, Yorik | — |
| 13 | Yorik rehearses the UML expiry sequence; fallback is B15 | Yorik | 16 |
| — | Alarm e-mail confirmation before the final release (#515) | Liu Jin | 5 |

## Effort per member

Estimated from the planned hours of the issues each person delivered, credited to whoever actually did the work, and adjusted by the team. Total ≈ 50 person-days; each member confirms their figure.

| Member | Person-days | Main area |
|---|---|---|
| Liu Jin | 12.5 | AWS stacks, pipeline, service split |
| Panpakorn (Pan) | 11.5 | Messaging, scaling, login, load tests |
| Yu Qianshuo (Yorik) | 9 | Event outbox and relay, failure drills |
| Lim Jiechao | 8.5 | Lead; AWS accounts and guardrails; architecture study and proof of concept; presentation |
| Wang Ziang (Eric) | 8.5 | Notification consumer, dead-letter redrive, demo lead |

# A Appendix — operator commands

Run from an operator environment with the reviewed checkout (see the environment session runbook). Never paste outputs containing parameter values into chat or onto the shared screen.

### A.1 Nothing queued that could destroy the session

```
gh run list --repo PPsyrius/iss-groupbuy-team8 --workflow swe5001-env.yml --limit 5
gh run list --repo PPsyrius/iss-groupbuy-team8 --workflow swe5001-deploy.yml --limit 3
```

### A.2 Bring the environment up (release digests) and check health

```
gh workflow run swe5001-env.yml --repo PPsyrius/iss-groupbuy-team8 --ref prod -f action=up
curl -s https://d3lvq02o2e7tcn.cloudfront.net/api/actuator/health
aws application-autoscaling describe-scalable-targets --service-namespace ecs \
  --query 'ScalableTargets[].{id:ResourceId,min:MinCapacity,max:MaxCapacity,suspended:SuspendedStat
e}'
```

### A.3 V5 self-healing — stop one commerce task under load (not used in the demo)

```
aws ecs list-tasks --cluster blocbuy-swe5001 --service-name commerce-svc
aws ecs stop-task --cluster blocbuy-swe5001 --task <task-arn> --reason "V5 drill"
aws ecs describe-services --cluster blocbuy-swe5001 --services commerce-svc \
  --query 'services[0].events[:5]'
```

### A.4 Identity outage drill — stop and restore (not used in the demo)

```
aws ecs update-service --cluster blocbuy-swe5001 --service identity-svc --desired-count 0
aws ecs update-service --cluster blocbuy-swe5001 --service identity-svc --desired-count 1
```

### A.5 Demo admin and startable listings

```
infra/swe5001/scripts/grant-admin.sh '<demo google email>'
BLOCBUY_BASE_URL=https://d3lvq02o2e7tcn.cloudfront.net/api/v1 \
  BLOCBUY_TOKEN=<cookie value> scripts/seed-dev-demo-listings.sh
```

### A.6 V7 poison message

Use the drill script delivered by #459 if it exists. Otherwise publish one malformed envelope to the topic with the routing attribute the notification subscription filters on (read the filter policy first; a message without it is filtered out and never reaches the queue):

```
TOPIC=$(aws sns list-topics --query "Topics[?ends_with(TopicArn, ':blocbuy-swe5001-domain-events')].
TopicArn" --output text)
aws sns list-subscriptions-by-topic --topic-arn "$TOPIC"     # find the notification subscription
aws sns get-subscription-attributes --subscription-arn <arn> --query 'Attributes.FilterPolicy'
aws sns publish --topic-arn "$TOPIC" --message '{"malformed":true}' \
  --message-attributes '{"eventType":{"DataType":"String","StringValue":"PaymentSucceeded"}}'
```

Add any further attribute the filter policy requires. With a 60 s visibility timeout and maxReceiveCount 5, expect the DLQ after about 5–6 minutes and the alarm a few minutes later.

### A.7 V3 load (not used in the talk; the 19 October procedure, for reference)

Load machine, POSIX shell, repository root. Ramp: 20% → 100% of KNEE_RPS in 5 min, to 300% in 10 min, hold 10 min, down 5 min (30 min total). The k6 preflight accepts only a stable one-task commerce deployment with scaling bounds 1..6 (`perf/k6/lib.mjs`), so the standing floor of 2 is lowered first, as on 19 October.

(a) Operator signed in through Identity Center with administrator rights: lower the commerce floor to 1 and wait for a stable 1/1 deployment.

```
aws application-autoscaling register-scalable-target --service-namespace ecs \
  --resource-id service/blocbuy-swe5001/commerce-svc --scalable-dimension ecs:service:DesiredCount \
  --min-capacity 1 --max-capacity 6
aws ecs update-service --cluster blocbuy-swe5001 --service commerce-svc --desired-count 1
aws ecs wait services-stable --cluster blocbuy-swe5001 --services commerce-svc
```

(b) Load machine: capture the V3 preflight in the clean reviewed checkout whose HEAD becomes TEST_SHA, with the operator's AWS environment (`perf/k6/README.md`). It is read-only, and k6 rejects it if it is more than five minutes old when the load starts.

```
results=$(mktemp -d); chmod 700 "$results"
umask 077
bash infra/swe5001/scripts/http-preflight.sh v3 > "$results/preflight.json"
```

(c) Start the load within five minutes of the preflight.

```
export BASE_URL=https://d3lvq02o2e7tcn.cloudfront.net CONFIRM_TARGET=https://d3lvq02o2e7tcn.cloudfro
nt.net
export TEST_SHA=$(git rev-parse HEAD) IMAGE_SHA='<deployed commerce source sha>'
export EXPECTED_IMAGE_DIGEST='sha256:<deployed commerce digest>' RUN_LABEL=20261021-v3-presentation
docker run --rm --user "$(id -u):$(id -g)" \
  --mount "type=bind,source=$PWD/perf/k6,target=/scripts,readonly" \
  --mount "type=bind,source=$results,target=/results" \
  --env BASE_URL --env CONFIRM_TARGET --env TEST_SHA --env IMAGE_SHA --env EXPECTED_IMAGE_DIGEST --e
nv RUN_LABEL \
  grafana/k6:1.6.1@sha256:a5ad6bc089a08d77c3ec49f3db8c6fa7a148e4073efcac44c675dbaf3568d8e1 run \
  --env MODE=v3 --env TOPOLOGY=distributed --env KNEE_RPS=200 \
  --env PREFLIGHT_PATH=/results/preflight.json --env SUMMARY_PATH=/results/summary.json \
  --out json=/results/samples.jsonl /scripts/browse.js
```

Afterwards, set the commerce floor back to 2, or let A.9 `down` remove the runtime:

```
aws application-autoscaling register-scalable-target --service-namespace ecs \
  --resource-id service/blocbuy-swe5001/commerce-svc --scalable-dimension ecs:service:DesiredCount \
  --min-capacity 2 --max-capacity 6
```

### A.8 V4 burst (not used in the demo; requires the reviewed v4_demo_enabled=true deployment)

```
export V4_RUN_LABEL=v4-20261021-presentation V4_RECIPIENT_ID=<already-active synthetic identity uuid
>
export EXPECTED_IMAGE_DIGEST=sha256:<commerce digest> EXPECTED_NOTIFICATION_DIGEST=sha256:<notificat
ion digest>
```

```
export CONFIRM_V4_SEED=2000 V4_RESULTS_DIR=<new empty private directory outside the checkout>
bash infra/swe5001/scripts/v4-burst.sh
```

Restore `v4_demo_enabled=false` with a reviewed apply after the talk (or let `down` remove the runtime).

### A.9 Tear down and verify

```
gh workflow run swe5001-env.yml --repo PPsyrius/iss-groupbuy-team8 --ref prod -f action=down
sleep 5   # let the run appear
RUN=$(gh run list --workflow swe5001-env.yml --repo PPsyrius/iss-groupbuy-team8 --branch prod --limi
t 1 --json databaseId -q ".[0].databaseId")
gh run watch "$RUN" --repo PPsyrius/iss-groupbuy-team8 --exit-status   # status has its own concurre
ncy group: wait until down has succeeded
gh workflow run swe5001-env.yml --repo PPsyrius/iss-groupbuy-team8 --ref prod -f action=status
```

# B Appendix — backup slides (Q&A only)

The backups follow slide 24 in the HTML deck and the PDF. In the deck, press **B** for the index (BX) and click a row; slide 24 stays up otherwise. Each backup gives the AD and AISS IDs in the Q&A answers a slide to point to.

| ID | Slide | Answers | Owner | What to say |
|---|---|---|---|---|
| B1a | Logical decisions (AD-02 to AD-06) | Why did you choose X over Y? (logical) | Eric | Read the decision, then the reason. AD-02 is slide 7; AD-04 to AD-06 are the cards on slide 8. |
| B1b | Physical decisions (AD-07 to AD-14) | Why Fargate, a NAT instance, one database server? | Liu Jin | Costs are AWS list prices for ap-southeast-1; the US$125/month always-on figure is the proposal estimate. |
| B2 | Known limitations and the way out | Single points of failure, a large commerce service | Liu Jin | Name the limitation plainly, then the resolution path. Single-zone database and cache is AISS-01. |
| B3 | Context map: where service boundaries may go | What does DDD actually change? | Eric | Four contexts share one service because they share transactions. Anti-corruption layers sit in front of Google and Stripe; notification only hears published events. |
| B4 | Aggregates and their life cycles | How do a team and an order change state? | Eric | A team can move only once, from OPEN. FAILED and CANCELED are allowed by the status table but nothing in the code triggers them: say so if asked. |
| B5 | Layers, and tests that keep boundaries in place | How do you stop boundaries eroding? | Eric | 18 architecture rules run with the unit tests, so a call across a boundary fails the build. |
| B6 | Joining a team by paying, step by step | Show me the payment path, not expiry | Yorik | Signature is checked outside the transaction. One transaction then deduplicates, locks in a fixed order, admits the member, saves the order, takes stock and writes the outbox; the active user is re-checked inside it. |
| B7 | Technology and services | Exact services, sizes and versions | Liu Jin | Read only the row that was asked about. |
| B8 | Everything the real cloud taught us | What else went wrong? | Pan | Ten findings in order. The last open one is the paused-scaling repair, which has no fix yet. |
| B9 | Infrastructure as code, and the actual bill | Stacks, lifetimes and real cost | Pan | The whole-account bill is mostly the always-on development server; this project's runtime cost US$0.49. |
| B10 | API access for producers and consumers | Platform engineering: robust API access | Eric | Be upfront about what is not built: no API gateway, per-client keys or rate limiting; internal endpoints rely on network rules. |
| B11 | Analytics: what the platform measures | Analytics to improve the platform | Lim | Operational analytics are in use. Business analytics are possible from data we already store (share invites and referral credits), but no reports are built yet. |

| | | | | |
|---|---|---|---|---|
| B12 | Scale-out and resilience results | Evidence behind slides 5, 14 and 17 | Lim | Never upgrade a local result to an AWS one. Anything not run on AWS stays "not run on AWS" with its local result. |
| B13 | Logical view, simplified | Plainer companion to slide 9 | Eric | Same content as slide 9 without UML notation. |
| B14 | Physical view, simplified | Plainer companion to slide 13 | Liu Jin | Same content as slide 13 without UML notation. |
| B15 | Team expiry, numbered steps | Plainer companion to slide 16 | Yorik | Use the slide 16 wording; the step numbers match. |